ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



MẠNG MÁY TÍNH

BTL1: DISTRIBUTED AND STREAMING BASE LIENT-SERVER COLLABORATION APPLICATION

Giảng viên hướng dẫn: Bùi Xuân Giang

Lớp: L01

Nhóm: 8

TP. Hồ Chí Minh, tháng 10/2026

Đánh giá thành viên

STT	Thành viên nhóm	MSSV	Đánh giá
1	Phạm Nghĩa Trung	2413713	100%
2	Nguyễn Minh Bảo		100%
3	Đào Nhật Toàn		100%
4	Mai Hồng Phước		100%

Mục lục

1 Đặc tả Giao thức **3**
1.1 Cấu trúc Header Gói tin . . . 3
1.2 Các Loại Gói tin . . . 3
1.3 Các Cờ Điều khiển . . . 4
1.4 Giới hạn Payload & Phân mảnh . . . 4
1.5 Bảo mật & Nhận diện (Security & Identification) . . . 5
1.6 Quá trình đóng gói và mở gói tin . . . 5
1.6.1 Hàm `pack` . . . 5
1.6.2 Hàm `unpack` . . . 6
1.7 Quản lý vòng đời Kết nối . . . 6
1.7.1 Cơ chế trao đổi Khóa . . . 6
1.7.2 Cơ chế chuyển vùng địa chỉ . . . 7
1.7.3 Duy trì nhịp tim và đảm bảo độ tin cậy . . . 7

2 Server và Logic Định tuyến **9**
2.1 Kiến trúc tổng quát . . . 9
2.2 Event loop non-blocking và xử lý datagram . . . 9
2.3 NAT Rebinding và bảo vệ định tuyến . . . 10
2.4 Xác thực người dùng và quản lý online state . . . 10
2.5 Logic định tuyến broadcast và unicast . . . 11
2.6 Xử lý file và stream dữ liệu . . . 11
2.7 Tính sẵn sàng và tự xử lý lỗi . . . 12
2.8 Kết luận . . . 12

Tài liệu tham khảo **13**

1 Đặc tả Giao thức

Ứng dụng triển khai một giao thức truyền tải tin cậy tùy chỉnh trên nền UDP, với nguồn tham khảo từ Google QUIC. Giao thức này cung cấp khả năng quản lý trạng thái kết nối, độ tin cậy dữ liệu và mã hóa payload.

1.1 Cấu trúc Header Gói tin

Mỗi gói tin truyền qua mạng đều được đóng gói kèm một header cố định kích thước 22 byte. Header này được tuần tự hóa theo thứ tự byte mạng Big-Endian để đảm bảo tính tương thích chéo giữa các nền tảng hệ điều hành.

Trường (Field)	Kích thước	Kiểu	Mô tả
`msg_type`	1 byte	`uint8`	Xác định loại gói tin (VD: DATA, ACK).
`conn_id`	4 byte	`uint32`	Định danh duy nhất cho kết nối logic.
`seq_num`	4 byte	`uint32`	Số thứ tự dùng cho quản lý độ tin cậy và truyền lại.
`flags`	1 byte	`uint8`	Cờ điều khiển dạng bitmask (VD: EOM, ENCRYPTED).
`stream_id`	2 byte	`uint16`	Định danh luồng dữ liệu ghép kênh (multiplexed).
`offset`	4 byte	`uint32`	Vị trí byte (offset) trong luồng để ráp lại dữ liệu.
`payload_len`	2 byte	`uint16`	Chiều dài của dữ liệu ứng dụng (payload) đính kèm.
`reserved`	4 byte	`uint32`	Dự phòng cho tương lai (căn lề cho đủ 22 byte).

Bảng 1: *Bố cục Header 22-byte của Giao thức Tùy chỉnh*

1.2 Các Loại Gói tin

Trường `msg_type` quyết định luồng xử lý và định tuyến của gói tin. Giao thức định nghĩa các loại thông điệp cốt lõi sau:

- `0x01 PKT_HELLO`: Khởi tạo quá trình bắt tay bảo mật (Client gửi Server).
- `0x02 PKT_HELLO_ACK`: Phản hồi quá trình bắt tay (Server phản hồi Client).
- `0x03 PKT_DATA`: Chuyên chở dữ liệu ứng dụng (payload).
- `0x04 PKT_ACK`: Gói tin xác nhận chọn lọc (Selective Acknowledgment - SACK) báo đã nhận thành công các số thứ tự gói.

- `0x05 PKT_HEARTBEAT`: Gói tin ping giữ mạng (keepalive) để duy trì cổng NAT/Firewall.
- `0x06 PKT_HEARTBEAT_ACK`: Phản hồi pong cho gói tin giữ mạng.
- `0x07 PKT_CLOSE`: Kích hoạt quá trình đóng kết nối an toàn.
- `0x08 PKT_AUTH_REQ` & `0x09 PKT_AUTH_RESP`: Trao đổi thông tin xác thực ở tầng ứng dụng.

1.3 Các Cờ Điều khiển

Trường `flags` sử dụng cơ chế bitmask để báo hiệu các trạng thái cụ thể của gói tin mà không làm tốn thêm dung lượng:

- `0x01 FLAG_EOM`: End-of-Message. Đánh dấu đây là mảnh dữ liệu cuối cùng của một thông điệp/file bị phân mảnh.
- `0x02 FLAG_FIN`: Kết thúc luồng. Báo hiệu sẽ không có thêm dữ liệu nào được truyền trên luồng này nữa.
- `0x04 FLAG_SYN`: Khởi tạo luồng.
- `0x08 FLAG_ENCRYPTED`: Khẳng định payload đính kèm đã được bảo vệ bằng thuật toán mã hóa AES-256-GCM.

1.4 Giới hạn Payload & Phân mảnh

Để giảm thiểu rủi ro bị phân mảnh ở tầng IP và rớt gói tin trên mạng lưới, giao thức áp đặt các ranh giới Kích thước Truyền tải Tối đa nghiêm ngặt:

- **Tải trọng UDP Tối đa:** 1400 byte.
- **Chi phí Header:** 22 byte.
- **Tải trọng Ứng dụng Tối đa:** 1378 byte trên mỗi gói tin.

Các khối dữ liệu vượt quá ngưỡng 1378 byte sẽ được module `Stream` tự động băm nhỏ ở bên gửi và ráp lại tại đích đến thông qua việc kết hợp tham số `offset` và cờ `FLAG_EOM`.

1.5 Bảo mật & Nhận diện (Security & Identification)

- **Số Ma thuật của Giao thức (Magic Number):** Một chữ ký nhận diện 4-byte (`0xDEADBEEF`) được định nghĩa để loại bỏ lưu lượng rác mạng hoặc các trình quét cổng (port scanners) một cách nhanh chóng trước khi tiến hành phân tích sâu gói tin.

- **Mã hóa Payload:** Các gói tin dữ liệu (`PKT_DATA`) hỗ trợ mã hóa mạnh mẽ bằng bộ mã hóa chuẩn công nghiệp AES-256-GCM. Quá trình này sử dụng một Khóa phiên dùng chung (Session Key) 32-byte (256-bit), một Nonce 12-byte (96-bit) và một Thẻ xác thực (Authentication Tag) 16-byte (128-bit) để đảm bảo đồng thời tính bảo mật và tính toàn vẹn của dữ liệu.

1.6 Quá trình đóng gói và mở gói tin

Trong kiến trúc giao thức truyền tải UDP tin cậy, hai hàm `pack` và `unpack` đóng vai trò là cửa ngõ giao tiếp giữa các cấu trúc dữ liệu ở tầng ứng dụng và môi trường truyền dẫn vật lý (luồng byte nhị phân thô). Quá trình này đảm bảo mọi thông tin điều khiển và dữ liệu đều được chuyển đổi theo một định dạng cấu trúc đồng nhất theo kiểu Big-Endian.

1.6.1 Hàm `pack`

- Mục đích: Hàm `pack` chịu trách nhiệm đóng gói các dữ liệu điều khiển (metadata) và dữ liệu ứng dụng (payload) thành một chuỗi byte (byte sequence) duy nhất, sẵn sàng để gửi qua UDP socket.

- Cơ chế hoạt động và Tác dụng:

  - Hàm nhận vào các tham số đại diện cho các trường của Header như: loại tin nhắn (`msg_type`), định danh kết nối (`conn_id`), số thứ tự (`seq_num`), các cờ điều khiển (`flags`), ...
  - Tham số `payload_len` được tính toán tự động thông qua hàm `len(payload)`, giúp loại bỏ nguy cơ sai sót do truyền nhầm chiều dài.
  - Sử dụng hàm `struct.pack` của Python với khuôn mẫu định dạng `_HEADER_FMT` để ép kiểu các số nguyên thành một khối nhị phân chính xác 22 byte.
  - Cuối cùng, khối Header 22 byte này được nối trực tiếp với mảng byte của phần `payload` để tạo thành một gói tin hoàn chỉnh (`header_bytes + payload`).

1.6.2 Hàm `unpack`

- Mục đích: Hàm `unpack` thực hiện quá trình giải đóng gói, tiếp nhận luồng byte thô từ tầng mạng và bóc tách chúng ngược lại thành đối tượng cấu trúc `Header` và khối dữ liệu `payload` để tầng ứng dụng dễ dàng xử lý.
- Cơ chế hoạt động và Tác dụng:
  - Kiểm tra an toàn: Bước đầu tiên, hàm kiểm tra độ dài của luồng dữ liệu (`len(raw_bytes)`). Nếu chuỗi byte nhỏ hơn ngưỡng tối thiểu `_HEADER_SIZE` (22 byte), hàm sẽ chủ động ném ra ngoại lệ `ValueError`.
  - Trích xuất Header: Hàm tiến hành cắt đúng 22 byte đầu tiên của luồng mạng (`raw_bytes[:_HEADER_SIZE]`) và đưa vào `struct.unpack` để phân rã thành 8 trường dữ liệu độc lập. Các trường này sau đó được bọc vào một đối tượng `Header` (sử dụng `NamedTuple`) để việc truy xuất trong mã nguồn được tường minh
  - Trích xuất Payload an toàn: Dựa vào thông số `payload_len` vừa giải mã được từ Header, hàm sẽ cắt chính xác phần dữ liệu ứng dụng nằm ngay sau Header (`raw_bytes[_HEADER_SIZE : _HEADER_SIZE + header.payload_len]`). Cách cắt mảng này giúp hệ thống chỉ lấy đúng phần dữ liệu thực tế, tự động loại bỏ các byte rác thừa lọt vào trong quá trình truyền qua mạng

1.7 Quản lý vòng đời Kết nối

Lớp `Connection` (nằm trong `transport/connection.py`) đóng vai trò là lõi trung tâm của giao thức, chịu trách nhiệm nâng cấp một socket UDP không trạng thái thành một kênh truyền thông tin cậy, có trạng thái và bảo mật. Nó điều phối sự tương tác giữa các module mã hóa, quản lý luồng và kiểm soát độ tin cậy

1.7.1 Cơ chế trao đổi Khóa

Mục đích của nhóm hàm bắt tay là thiết lập một kênh truyền thông an toàn mà không để lộ khóa bí mật qua mạng, ngăn chặn các cuộc tấn công nghe lén (Eavesdropping). Quá trình này ứng dụng thuật toán Diffie-Hellman trên đường cong elliptic (X25519).

- Khởi tạo từ Client (`handle_handshake`): Hàm này tạo một cặp khóa bất đối xứng cục bộ. Khóa công khai được chuyển thành chuỗi Hex, bọc trong cấu trúc JSON và đóng gói dưới dạng thông điệp `PKT_HELLO` để gửi tới Server.

- Tiếp nhận tại Server (`_handle_hello`): Khi nhận được `PKT_HELLO`, Server giải mã JSON để lấy Khóa công khai của Client. Server tiếp tục tự sinh một cặp khóa mới và sử dụng khóa bí mật của mình kết hợp với khóa công khai của Client để sinh ra khóa phiên dùng chung (`_session_key`). Cuối cùng, Server gửi trả khóa công khai của chính nó thông qua thông điệp `PKT_HELLO_ACK`.
- Hoàn tất tại Client (`_handle_hello_ack`): Client nhận Khóa công khai của Server và áp dụng toán học tương tự để tự tính ra `_session_key`. Tại thời điểm này, sự kiện `handshake_complete` được kích hoạt (`.set()`), cho phép dữ liệu ứng dụng bắt đầu được mã hóa AES-256-GCM và truyền đi an toàn.

1.7.2 Cơ chế chuyển vùng địa chỉ

Khác với TCP (dựa trên bộ 4 giá trị Source/Dest IP và Port), giao thức này định danh kết nối qua `conn_id`. Điều này cho phép một thiết bị duy trì kết nối ngay cả khi chuyển mạng Hàm `verify_migration_datagram`: Nhằm ngăn chặn các cuộc tấn công giả mạo IP nhằm chiếm quyền điều khiển kết nối, hàm này yêu cầu xác thực nghiêm ngặt. Khi Server nhận thấy có dữ liệu từ một IP/Port lạ mang cùng một `conn_id` cũ, nó không chấp nhận ngay. Thay vào đó, nó buộc gói tin này phải là gói `PKT_DATA` đã được mã hóa (`FLAG_ENCRYPTED`). Nếu hệ thống có thể giải mã thành công gói tin này bằng `_session_key` cũ, chứng tỏ IP lạ này thực chất vẫn là Client hợp lệ đang giữ khóa. Lúc này, Server mới cập nhật địa chỉ `remote_addr` mới.

1.7.3 Duy trì nhịp tim và đảm bảo độ tin cậy

Mạng lưới trung gian (như NAT/Firewall) thường tự động đóng các cổng UDP nếu không có dữ liệu lưu thông trong một thời gian ngắn. Do đó, kết nối cần được giám sát liên tục.

- Giữ cổng mạng (`_send_heartbeat` & `_send_heartbeat_ack`): Các hàm này gửi các gói tin có tải trọng rỗng (`payload=b""`) qua lại giữa hai đầu cuối. Hoạt động này nhằm mục đích duy trì bảng định tuyến NAT của nhà mạng luôn mở, đồng thời xác nhận đối phương vẫn đang trực tuyến
- Đồng hồ Sinh học (`tick`): Đây là hàm được kích hoạt định kỳ (khoảng 50ms/lần) bởi vòng lặp sự kiện chính. Hàm thực hiện 3 nhiệm vụ cốt lõi:
    1. Kiểm soát Timeout: Nếu khoảng thời gian kể từ gói tin cuối cùng nhận được vượt quá `IDLE_TIMEOUT` (ví dụ 30 giây), hệ thống kết luận đối phương đã mất mạng hoặc treo ứng dụng, tự động đánh dấu `closed = True` để giải phóng tài nguyên bộ nhớ.

2. Phát nhịp tim: Kích hoạt hàm `_send_heartbeat` nếu đã tới chu kỳ `HEARTBEAT_INTERVAL`.
3. Truyền lại: Liên kết với module `ReliabilityManager` để lấy danh sách các gói tin chưa được xác nhận (SACK) đã quá hạn (RTO Timeout). Sau đó, thực hiện gọi `_raw_send` để bắn lại các gói tin này xuống mạng, đảm bảo không một mảnh dữ liệu nào bị thất thoát.

3 Server và Logic Định tuyến

3.1 Kiến trúc tổng quát

Mô hình Server của hệ thống được xây dựng theo hướng **single-entry dispatcher** trên giao thức UDP, trong đó tất cả các gói tin từ client đều đi qua một socket UDP duy nhất và được xử lý bởi event loop dựa trên *selectors*. Cấu trúc này cho phép server quản lý đồng thời nhiều phiên làm việc mà không cần mở một socket riêng cho từng client, giúp tiết kiệm tài nguyên và tăng khả năng mở rộng lên hàng chục client cùng lúc.

Ở tầng trung tâm, server giữ ba cấu trúc dữ liệu quan trọng:

- `_sessions`: ánh xạ từ `conn_id` sang đối tượng `Session`, đại diện cho trạng thái của mỗi client.
- `_addr_to_conn`: ánh xạ từ địa chỉ IP/port của client sang `conn_id`, hỗ trợ phát hiện NAT rebinding và cập nhật vị trí client.
- `_user_to_conn`: ánh xạ từ username sang kết nối đang hoạt động, là cơ sở để thực hiện định tuyến tin nhắn cá nhân.

Mỗi client khi kết nối tới server sẽ được tạo một `Session` riêng, chứa thông tin xác thực, trạng thái online, và callback để thực hiện broadcast/unicast. Cách tổ chức này đảm bảo rằng lỗi của một client không làm sụp toàn bộ server vì logic xử lý được cô lập theo từng phiên.

3.2 Event loop non-blocking và xử lý datagram

Server khởi tạo socket UDP ở chế độ non-blocking bằng cách gọi:

```
self._sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
self._sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
self._sock.bind((host, port))
self._sock.setblocking(False)
self._sel.register(self._sock, selectors.EVENT_READ)
```

Sau đó, server chạy vòng lặp chính dựa trên `selectors.select(timeout=TICK_INTERVAL)`. Mỗi khi socket sẵn sàng đọc dữ liệu, server gọi hàm `handle_datagram(data, addr)` để phân tích packet và chuyển tiếp tới session tương ứng. Cơ chế này cho phép server xử lý đồng thời nhiều sự kiện I/O mà không cần tạo thread cho từng kết nối, nhờ đó hiệu quả hơn so với mô hình blocking socket.

Hàm `handle_datagram()` thực hiện những bước chính sau:

1. Giải mã header để lấy `conn_id` và loại tin nhắn.
2. Kiểm tra session tương ứng đã tồn tại chưa. Nếu chưa, server tạo một session mới.
3. Kiểm tra khả năng **NAT Rebinding**: nếu client đổi địa chỉ IP/port nhưng gói tin hợp lệ, server cập nhật `remote_addr`; nếu không hợp lệ, server từ chối cập nhật.
4. Chuyển dữ liệu tới tầng transport bằng `session.connection.receive_packet(data)`.
5. Nếu nhận được gói đóng kết nối, đánh dấu session bị đóng và chuẩn bị rời khỏi hệ thống.

3.3 NAT Rebinding và bảo vệ định tuyến

Một vấn đề quan trọng trong môi trường UDP là client có thể di chuyển giữa các mạng hoặc đổi port tạm thời do NAT. Để tránh hacker hoặc client giả mạo đổi hướng routing, server không cập nhật địa chỉ client một cách tùy tiện. Thay vào đó, nó chỉ chấp nhận rebinding khi gói dữ liệu đi kèm được xác thực bởi cơ chế AEAD của tầng transport.

Trong quá trình này, server kiểm tra các điều kiện sau:

- Địa chỉ cũ và địa chỉ mới phải là cùng một `conn_id`.
- Địa chỉ mới không thuộc sở hữu của một client khác đang hoạt động.
- Gói tin gửi đến từ địa chỉ mới phải đi qua xác thực migration datagram hợp lệ.

Nếu điều kiện không thỏa, server ghi log cảnh báo và từ chối thay đổi địa chỉ đến, tránh rò rỉ dữ liệu hay gán sai session. Đây là biện pháp bảo vệ quan trọng nhằm đảm bảo tính toàn vẹn của định tuyến khi có NAT hoặc chuyển mạng.

3.4 Xác thực người dùng và quản lý online state

Ở tầng ứng dụng, mỗi session triển khai hàm `process_payload()` để parse JSON từ client. Dữ liệu đầu vào được kiểm tra cực kỳ chặt chẽ:

- Bắt buộc là JSON object hợp lệ.
- Loại tin nhắn phải nằm trong tập hợp `AUTH`, `MSG`, `FILE`, `FILE_DATA`, `LOGOUT`.
- Nếu chưa đăng nhập mà gửi các tin nhắn không thuộc AUTH, server trả về lỗi `ERROR`.

Với loại tin nhắn `AUTH`, hàm `verify_auth()` sẽ kiểm tra username và password hash với SQLite. Nếu thành công, session được đánh dấu là `authenticated = True` và lưu username vào bộ nhớ. Điều này cho phép server xây dựng bảng ánh xạ `username -> conn_id` để định tuyến trực tiếp đến user đích.

Việc xác thực được thực hiện dạng "đóng gói" và an toàn vì server:

- giới hạn độ dài username và mật khẩu hash;
- đảm bảo hash chỉ gồm hex chữ số thập lục phân;
- ngăn chặn chuyển tài khoản khi session đã authenticated;
- phát hành thông báo `AUTH_RESP` và user join notification cho các client còn lại.

3.5 Logic định tuyến broadcast và unicast

Sau khi user đăng nhập thành công, server đảm nhiệm việc định tuyến tin nhắn đến đúng đích. Hai hàm cốt lõi là `_broadcast()` và `_unicast()`.

Broadcast: Khi target là `"all"`, server lặp qua tất cả session đang online, kiểm tra điều kiện xác thực và trạng thái đóng, rồi gửi tin nhắn tới từng client. Vì hàm này loại trừ sender bằng tham số `exclude`, nên tin nhắn không bị lặp lại cho người gửi. Đây là cơ chế chính để triển khai phòng chat cộng đồng.

Unicast: Khi target là một username cụ thể, server tra cứu trong `_user_to_conn`. Nếu target online, server ghi dữ liệu trực tiếp vào stream tương ứng. Nếu không online hoặc không tồn tại, hệ thống trả về lỗi và không làm gián đoạn các session khác. Cách thiết kế này đảm bảo dữ liệu chỉ đến đúng recipient, không rò rỉ sang client khác.

Sơ đồ logic định tuyến có thể biểu diễn như sau:

$$\text{target} = \begin{cases} \text{all}, & \text{nếu } msg.target = \text{“all”} \\ \text{user_map}[msg.target], & \text{nếu } msg.target \in \text{username} \end{cases} \tag{1}$$

Đây chính là cơ chế làm cho hệ thống gửi tin nhắn từ A sang B không bị rò rỉ sang C, đồng thời vẫn hỗ trợ broadcast cho cả phòng chat.

3.6 Xử lý file và stream dữ liệu

Ngoài tin nhắn văn bản, hệ thống còn hỗ trợ truyền file giữa các client. Trong mô hình này, file không được gửi nguyên vẹn như một khối lớn mà được chia thành các chunk theo stream ID đã xác thực. Server xử lý ba loại thông điệp:

- `FILE`: bắt đầu/khởi tạo transfer, xác thực filename, size, sha256, stream_id.

- `FILE_DATA`: chứa offset và dữ liệu chunk.
- `FILE_READY`: báo nhận metadata và hợp lệ cho file transfer.

Mỗi transfer được theo dõi trong `_file_transfers` với cấu trúc `(target, expected_size, received_size)`. Khi nhận được chunk mới, server kiểm tra:

- stream_id có hợp lệ không;
- offset không âm và không vượt quá giới hạn;
- độ dài dữ liệu chunk không quá ngưỡng cho phép;
- tổng kích thước chunk nhận về không vượt quá kích thước file khai báo.

Nếu bất kỳ điều kiện nào sai, server sẽ từ chối chunk và ghi log cảnh báo. Cơ chế này bảo vệ server khỏi tấn công bằng dữ liệu bất thường, đồng thời đảm bảo file đến đúng đích trong thời gian hợp lệ.

3.7 Tính sẵn sàng và tự xử lý lỗi

Server không chỉ nhận dữ liệu mà còn duy trì vòng lặp tick nền chạy định kỳ bằng `_tick_loop()`. Trong mỗi chu kỳ, server gọi `session.connection.tick()` để xử lý keepalive, đảm bảo socket hoạt động ổn định, và phát hiện connection đã đóng hoặc time-out. Nếu một client sập hoặc bị ngắt, session liên quan được xóa sạch khỏi `_sessions`, mapping user tương ứng bị xoá, và toàn bộ người còn lại được cập nhật danh sách online mới bằng thông báo `USER_LIST`.

Với cách thiết kế này, một client lỗi không làm crash server. Chỉ session đó bị đóng, các session khác vẫn hoạt động bình thường. Đây là nguyên tắc quan trọng trong hệ thống phân tán có số lượng client đồng thời từ 10 đến 20 và cần đảm bảo độ tin cậy tốt trong môi trường UDP.

3.8 Kết luận

Server của Assignment 1 được thiết kế theo mô hình event-driven, non-blocking UDP với session-based routing. Kiến trúc này cho phép hệ thống xử lý nhiều client đồng thời, xác thực an toàn, định tuyến chat/broadcast/file theo đúng người nhận, và tự động xử lý các trường hợp NAT rebinding, client mất kết nối, hoặc dữ liệu sai định dạng. Chính vì vậy, server không chỉ đáp ứng yêu cầu kỹ thuật của bài toán mà còn thể hiện được tính tận dụng tốt của I/O multiplexing, thread safety và logic định tuyến phân tán thực tế.

Tài liệu tham khảo

[1] A. Langley, A. Riddoch, A. Wilk, A. Vicente, C. Krasic, D. Zhang, F. Yang, F. Kouranov, I. Swett, J. Iyengar *et al.*, "The quic transport protocol: Design and internet-scale deployment," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2017, pp. 183–196.

[2] J. Iyengar and M. Thomson, "Quic: A udp-based multiplexed and secure transport," Internet Engineering Task Force (IETF), RFC 9000, May 2021. [Online]. Available: https://www.rfc-editor.org/info/rfc9000

[3] M. Thomson and S. Turner, "Using tls to secure quic," Internet Engineering Task Force (IETF), RFC 9001, May 2021. [Online]. Available: https://www.rfc-editor.org/info/rfc9001

[4] J. Iyengar and I. Swett, "Quic loss detection and congestion control," Internet Engineering Task Force (IETF), RFC 9002, May 2021. [Online]. Available: https://www.rfc-editor.org/info/rfc9002

[5] Cloudflare, "quiche: A robust, low-latency quic implementation in rust," https://github.com/cloudflare/quiche, 2024, open source.

[6] quic-go Team, "quic-go: A quic implementation in pure go," https://github.com/quic-go/quic-go, 2024, open source.

[7] Google, "Chromium quic source code (c++)," https://source.chromium.org/chromium/chromium/src/+/main:net/quic/, 2024, open source.